

3 – Création des tables

Création d'une table

- **CREATE TABLE** : Pour créer une table en donnant son nom, ses attributs et ses contraintes.
- CREATE TABLE <nom table>
 (<nom attribut> <type attribut> [DEFAULT <expression>]
 [<contrainte(s) d'attribut>], ... ,
 [<contrainte(s) de table>]);
- CREATE TABLE <nom table> [(definition)] AS <requête>;
 - Attention ne prend que les données (pas les contraintes, ni les index...).
- **DEFAULT** <expression> : Possibilité d'attribuer une valeur par défaut à un attribut.

Exemple

```
CREATE TABLE PILOTES_NICOIS
AS
    SELECT PLNUM, PLNOM, PLPRENOM
    FROM PILOTE
    WHERE VILLE='NICE' ;
```

- Ex : Création des tables COMMANDES et DETAILS_COMMANDES (sans aucune contrainte) :

```
CREATE TABLE COMMANDES (
    NO_COMMANDE INT,
    DATE_COMMANDE DATE,
    CODE_CLIENT CHAR(5) ) ;
```

```
CREATE TABLE DETAILS_COMMANDES (
    NO_COMMANDE INT,
    REF_PRODUIT INT,
    QUANTITE INT DEFAULT 1) ;
```

Création d'une table

- <Nom table> : Le nom de la table doit être unique au niveau de la BD.
- <Nom attribut> : Deux attributs peuvent avoir le même nom dans la même base, mais pas dans la même table (utilisation de la notation pointée pour les distinguer).
- Conventions de nommage pour les noms de tables et d'attributs :
 - Pas vraiment de standard, dépend du SGBD, du langage, de l'OS, des habitudes de l'équipe... (lowercase, UPPERCASE, UpperCamelCase, lowerCamelCase, snake_case...).
 - PostgreSQL (lowercase), Oracle (UPPERCASE), MySQL sous Windows (insensible à la casse)...
 - Eviter les caractères spéciaux, les espaces, les mots réservés (ex : Date).
 - Choisir une convention et s'y tenir.

Types des attributs

- **Numériques** : INT, SMALLINT, BIGINT, DECIMAL(p, s), FLOAT, REAL, NUMBER(p,n)...
- **Chaîne de Caractères** : CHAR(), VARCHAR(), TEXT, LONGTEXT...
 - CHAR(n) : Chaîne de longueur fixe (n caractères).
 - VARCHAR(n) : Chaîne de longueur variable jusqu'à n caractères.
- **Dates** et **temps** : DATE, TIME, DATETIME, INTERVAL, TIMESTAMP...
- **Booléens** : BOOLEAN, TINYINT...
- Types ininterprétés (BLOB, BFILE, BINARY(n)...) : Stockage de données non structurées comme du texte, des images, des vidéos...
- ...

Types des attributs

- Choisir le type adapté aux données de la colonne.
 - Exemple : Utiliser SMALLINT pour des âges plutôt que INT.
 - Prévoir une taille suffisamment grande pour les noms (surtout si l'application est internationale).
- Quelques différences entre les SGBD :
 - [Oracle](#)
 - [MySQL](#)
 - [PostgreSQL](#)
 - [SQLite](#)
- Il est possible de créer des attributs auto-incrémentés.
 - Ex : `id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY`
- Certains SGBD proposent le type ENUM (sinon CHECK, cf diapos suivantes) :
 - Ex : `Catégorie ENUM('A','B','C')`

Contraintes d'intégrité

- On distingue les **contraintes d'attribut** qui ne font intervenir qu'un seul attribut (on peut en définir une ou plusieurs par attribut) des **contraintes de table** qui font intervenir plus d'un attribut.
- Les contraintes peuvent être optionnellement nommées en utilisant le préfixe **CONSTRAINT** :
 - Contrainte d'attribut :
 - <nom attribut> <type attribut> CONSTRAINT <nom contrainte> <expression de la contrainte>
 - Contrainte de table :
 - CONSTRAINT <nom contrainte> <expression de la contrainte>
 - Lorsqu'une contrainte est violée, plus facile pour la retrouver.
 - Permet aussi de les désactiver ou de les activer à la demande.

Contraintes d'intégrité

- Contrainte **CHECK**
- Contrainte d'attribut Contrainte de table

```
QTE INT CHECK(QTE>9)
CONSTRAINT CK_SAL CHECK(SAL+COMM<=5000)
```

- Contrainte **NOT NULL**

```
DATE_COMMANDE DATE NOT NULL
DATE_COMMANDE DATE NULL
```

- Contrainte **UNIQUE**

- Clé candidate, toutes les valeurs doivent être différentes (sauf NULL).

```
SOCIETE VARCHAR(40) UNIQUE
UNIQUE(NO_COMMANDE,REF_PRODUIT)
```

Contraintes d'intégrité

- Contrainte de **clé primaire**

- Identifiant de la relation (unique et non NULL).

```
NO_COMMANDE INT PRIMARY KEY  
PRIMARY KEY (NO_COMMANDE, REF_PRODUIT)
```

- Contrainte de **clé étrangère**

- Contrainte d'intégrité référentielle par rapport à une clé candidate ou primaire.

```
NO_COMMANDE INT REFERENCES COMMANDES (NO_COMMANDE)  
FOREIGN KEY (REF_PRODUIT) REFERENCES PRODUITS (REF_PRODUIT)  
FOREIGN KEY (A,B) REFERENCES TABLE (A,B)
```

- Rq :

- Le nom de l'attribut à la fin est optionnel s'il est identique
- Attention à l'ordre de création et de suppression des tables !

Contraintes d'intégrité

Nom	Prenom
Dupont	Jean
Martin	Louis
Dupont	Louise
...	...

- Attention, les 2 écritures suivantes ne sont pas équivalentes :
 - FOREIGN KEY (Nom, Prenom) REFERENCES Personnes(Nom, Prenom)
 - FOREIGN KEY (Nom) REFERENCES Personnes(Nom)
 - FOREIGN KEY (Prenom) REFERENCES Personnes(Prenom)
- => Faux car on pourrait avoir « Martin Jean » par exemple !

Contraintes d'intégrité

- Clause **ON DELETE** :
 - Permet de considérer le cas de la suppression d'une ligne référencée
 - ON DELETE CASCADE : suppression automatique de la ligne fille
 - ON DELETE SET NULL : clé étrangère à NULL
 - Sans la clause, la suppression d'une ligne référencée n'est pas autorisée
 - Il existe aussi ON UPDATE et SET DEFAULT.
- Ex : Lorsqu'une commande sera supprimée dans la table COMMANDES, cela entraînera automatiquement la suppression des lignes associées dans la table DETAILS_COMMANDES.

```
CREATE TABLE COMMANDES (  
  NO_COMMANDE INT, ... ) ;  
  
CREATE TABLE DETAILS_COMMANDES (  
  NO_COMMANDE INT,  
  ...  
  FOREIGN KEY (NO_COMMANDE) REFERENCES COMMANDES (NO_COMMANDE) ON DELETE CASCADE  
  ... ) ;
```

Modification d'une table

- ***ALTER TABLE*** : Pour modifier la définition d'une table.
 - Renommer une table, une colonne ou une contrainte
 - Ajouter une colonne ou une contrainte
 - Modifier une colonne ou une contrainte
 - Les valeurs d'une colonne modifiée sont mises à NULL pour chaque ligne existante si la colonne est redéfinie avec une plus petite largeur ou un type non compatible. Attention chaque SGBD traite le problème à sa manière.
 - Supprimer une colonne ou une contrainte
 - Modifier le type d'une colonne
 - ...

Exemple

```
ALTER TABLE DETAILS_COMMANDES RENAME TO DETAILS;
```

```
ALTER TABLE CLIENTS  
ADD COLUMN SOCIETE INT DEFAULT 1;
```

```
ALTER TABLE CLIENTS  
ADD PRIMARY KEY (CODE_CLIENT);
```

```
ALTER TABLE CLIENTS  
ALTER COLUMN SOCIETE TYPE VARCHAR(60);
```

```
ALTER TABLE CLIENTS  
RENAME COLUMN SOCIETE TO NOM;
```

```
ALTER TABLE CLIENTS  
DROP COLUMN TYPE;
```

```
ALTER TABLE DETAILS_COMMANDES  
ADD CONSTRAINT CK_QTE CHECK(QTE<20);
```

```
ALTER TABLE DETAILS_COMMANDES  
DROP CONSTRAINT CK_QTE;
```

Suppression d'une table

- ***DROP TABLE*** : Pour supprimer une table et tout son contenu (ainsi que les index associés).

```
DROP TABLE DETAILS_COMMANDES ;  
DROP TABLE COMMANDES ;  
DROP TABLE PRODUITS ;  
DROP TABLE CLIENTS ;
```

- Attention à l'ordre de suppression des tables (clés étrangères) => ordre inverse des créations.

Les vues

- **VIEW** : Pour donner à chaque utilisateur sa vision propre des données.
- Les vues sont des **tables virtuelles** dérivées de tables de base au moyen d'une requête SQL.
- La vue est perçue par l'utilisateur comme une table, mais la différence se fait au niveau du stockage des données. En effet, une vue n'est pas stockée physiquement, elle est évaluée à chaque fois qu'on en a besoin.
- Les vues s'utilisent dans les requêtes de la même façon que les tables. Les mises à jour sont effectuées dans les tables de base.

Les vues

■ Création d'une vue

- ❑ **CREATE VIEW** <nom vue> [(<attribut>, ...)] AS
<requête> [WITH CHECK OPTION];

```
CREATE VIEW VueSociete AS  
SELECT CODE_CLIENT, SOCIETE  
FROM CLIENTS;
```

- ❑ La liste facultative d'attributs permet de renommer les attributs qui vont être obtenus par la requête.
- ❑ WITH CHECK OPTION : Permet de contrôler les insertions au travers de vues, vérifie que la mise à jour ne produit que des lignes qui font partie de la sélection de la vue (conditions très restrictives dans la requête).

■ Supprimer une vue

- ❑ **DROP VIEW** <nom vue>;

Les vues

- Intérêts
 - Simplification de l'accès aux données en masquant les opérations de jointure.
 - Présentation de mêmes données sous différentes formes selon les utilisateurs.
 - Formuler des requêtes complexes qui nécessitent le passage par des vues.
 - Sauvegarde indirecte de requêtes complexes.
 - Renforcer la sécurité en masquant des lignes et des colonnes sensibles aux usagers non habilités.
 - Support de l'indépendance logique.
 - ...

Les vues

- Exemple de simplification de requête avec une vue
 - Soit la vue définie comme suit :

```
CREATE VIEW COMMANDES_SOCIETE AS
SELECT Co.NO_COMMANDE, DATE_COMMANDE, CODE_CLIENT, SOCIETE
FROM COMMANDES Co
JOIN CLIENTS Cl ON Co.CODE_CLIENT=Cl.CODE_CLIENT;
```

- La requête “Donner les informations des commandes de la société Rancho grande” se formule simplement à partir de la vue COMMANDES_SOCIETE :

```
SELECT NO_COMMANDE, DATE_COMMANDE
FROM COMMANDES_SOCIETE
WHERE SOCIETE='Rancho grande' ;
```

Les vues

- Exemple de support d'indépendance logique avec une vue
 - Suite à un remaniement physique, la table CLIENTS est éclatée en 2 tables :

```
CLIENTS1 (CODE_CLIENT, SOCIETE, CATEGORIE)
CLIENTS2 (CODE_CLIENT, ADRESSE, VILLE, CODE_POSTAL, PAYS)
```

- En créant la vue CLIENTS suivante, les requêtes qui utilisaient la table CLIENTS n'ont pas à être remaniées (les utilisateurs ne verront pas le changement) :

```
CREATE VIEW CLIENTS AS
SELECT C1.CODE_CLIENT, SOCIETE, ADRESSE, VILLE, CODE_POSTAL, PAYS, CATEGORIE
FROM CLIENTS1 C1
JOIN CLIENTS2 C2 ON C1.CODE_CLIENT=C2.CODE_CLIENT;
```

Insertion de données

- **INSERT INTO** : Pour insérer des données dans une table.
- Insertion d'une ligne :

```
INSERT INTO COMMANDES VALUES (10797, '2009-04-25', NULL) ;  
Ou :  
INSERT INTO COMMANDES (NO_COMMANDE, DATE_COMMANDE)  
VALUES (10797, '2009-04-25') ;
```

- Insertion de plusieurs lignes issues de la base :

```
INSERT INTO PILOTES_NICOIS (PLNUM, PLNOM, PLPRENOM)  
SELECT PLNUM, PLNOM, PLPRENOM  
FROM PILOTE  
WHERE VILLE='NICE' ;
```

Modification de données

- **UPDATE** : Pour modifier les valeurs d'une ou plusieurs colonnes, dans une ou plusieurs lignes existantes d'une table.

```
UPDATE PILOTE  
SET VILLE='LYON'  
WHERE PLNUM=4;
```

```
UPDATE PILOTE  
SET SALAIRE = SALAIRE+100  
WHERE VILLE='PARIS';
```

- Si la clause WHERE est omise, la modification porte sur la table entière.

Suppression de données

- **DELETE** : Pour supprimer une ou plusieurs lignes d'une table.

```
DELETE FROM PILOTE  
WHERE PLNUM=5 ;
```

```
DELETE FROM DETAILS_COMMANDES  
WHERE NO_COMMANDE = (SELECT NO_COMMANDE  
                     FROM COMMANDE  
                     WHERE CODE_CLIENT='ERNSH' ) ;
```

- L'omission de la clause WHERE entraîne l'élimination de toutes les lignes de la table !

Contrôle des transactions

- Une **transaction** est une suite d'opérations (lectures - écritures) effectuées sur une base de données, sous le contrôle d'un utilisateur en ligne. Correspond à un ensemble de **commandes LMD** (INSERT, UPDATE, DELETE).
 - Exemples :
 - Réservation d'une place d'avion
 - Retrait d'une somme d'argent à partir d'un distributeur...
 - Virement d'une somme d'argent d'un compte A vers un compte B...
- 2 actions possibles pour terminer une transaction :
 - **COMMIT** : Validation de la transaction.
 - **ROLLBACK** : Annulation de la transaction.
- Ne concerne pas les commandes LDD (CREATE, DROP, ALTER) : validation automatique.

Contrôle des transactions

```
SQL>select NOCDE, NOCLI, DATECDE,  
2 ETATCDE from COMMANDES ;
```

<u>NOCDE</u>	<u>NOCLI</u>	<u>DATECDE</u>	<u>ET</u>
1301	15	20/11/98	EC
1210	15	12/11/98	SO
1250	35	14/11/98	EC
1230	35	14/11/98	EC

```
SQL>insert into COMMANDES (NOCDE,  
2 NOCLI, DATECDE, ETATCDE)  
3 values (2000, 35, SYSDATE, 'EC') ;  
1 ligne créée.
```

```
SQL>select NOCDE, NOCLI, DATECDE,  
ETATCDE from COMMANDES ;
```

<u>NOCDE</u>	<u>NOCLI</u>	<u>DATECDE</u>	<u>ET</u>
1301	15	20/11/98	EC
1210	15	12/11/98	SO
1250	35	14/11/98	EC
1230	35	14/11/98	EC
2000	35	22/11/98	EC

```
SQL>rollback;
```

```
SQL>select NOCDE, NOCLI, DATECDE,  
ETATCDE from COMMANDES ;
```

<u>NOCDE</u>	<u>NOCLI</u>	<u>DATECDE</u>	<u>ET</u>
1301	15	20/11/98	EC
1210	15	12/11/98	SO
1250	35	14/11/98	EC
1230	35	14/11/98	EC

```
COMMIT;
```

La commande **ROLLBACK** défait l'opération de mise à jour.

La commande **COMMIT** valide définitivement les mises à jour. Plus aucune commande ROLLBACK ne pourra les annuler.

Contrôle des transactions

- Un SGBD relationnel doit respecter les propriétés ACID :
 - **Atomicité**
 - Une transaction s'effectue soit entièrement, soit pas du tout.
 - **Cohérence**
 - Les effets d'une transaction validée ne doivent jamais violer les contraintes d'intégrité de la BD.
 - **Isolation**
 - Une transaction ne doit jamais voir les résultats intermédiaires des autres transactions (propriété pouvant être relâchée).
 - **Durabilité**
 - Dès qu'une transaction est validée, le système doit assurer que ses effets ne seront jamais perdus (même en cas de panne).